



SQL Learning Guide

User Defined Functions & SQL Joins

Beginner-Friendly | Interview Ready | Practice Questions

Frontlines Edutech

www.frontlinesedutech.com

1. User Defined Functions (UDFs) in SQL

A User Defined Function (UDF) is a function created by the user to perform a specific task. Unlike built-in functions like SUM() or UPPER(), UDFs are custom-written to meet your specific business needs. Once created, they can be reused anywhere in your SQL queries — just like built-in functions.

There are three types of User Defined Functions in SQL Server:

- Scalar Function — returns a single value
- Table Valued Function (TVF) — returns a table
- Multi-Statement Table Valued Function (MTVF) — returns a table built through multiple steps

1.1 Scalar Function

Definition

A Scalar Function accepts zero or more input parameters and returns exactly ONE single value (like a number, text, or date). Think of it like a formula — you give it inputs, it gives back one answer.

Real-Life Analogy: Imagine a formula to calculate your annual bonus — you give your salary and performance rating, and the formula returns one number (your bonus amount). That's exactly what a scalar function does.

Syntax

```
CREATE FUNCTION function_name (@param1 datatype, @param2 datatype)
RETURNS return_datatype AS BEGIN      -- Your logic here      RETURN
value; END;
```

Example: Calculate Annual Salary

```
-- Create a scalar function to calculate annual salary
CREATE FUNCTION dbo.GetAnnualSalary (@MonthlySalary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @AnnualSalary DECIMAL(10,2);
    SET @AnnualSalary = @MonthlySalary * 12;
    RETURN @AnnualSalary;
END;

-- How to CALL / USE the function
SELECT EmpName,
       Salary AS MonthlySalary,
       dbo.GetAnnualSalary(Salary) AS AnnualSalary
FROM Employees;
```

Output:



EmpName	MonthlySalary	AnnualSalary
Alice	60000	720000
Bob	45000	540000
Carol	70000	840000

Key Points to Remember:

- Always use `dbo.FunctionName()` when calling the function (include the schema prefix)
- `RETURNS` keyword specifies what data type the function gives back
- `RETURN` inside `BEGIN...END` sends the final answer
- Scalar functions can be used in `SELECT`, `WHERE`, and `ORDER BY` clauses

1.2 Table Valued Function (TVF) — Inline

Definition

A Table Valued Function returns an entire TABLE as its output (not just one value). It is called 'Inline' because all the logic is written in a single SELECT statement inside RETURN. You can use it just like a regular table in your queries.

Real-Life Analogy: Imagine a filter in Excel — you tell it a department name, and it shows you a full table of employees from that department. An Inline TVF works the same way.

Syntax

```
CREATE FUNCTION function_name (@param datatype) RETURNS TABLE AS RETURN
(
    SELECT column1, column2 FROM TableName WHERE condition );
```

Example: Get Employees by Department

```
-- Create a TVF to get employees from a specific department
CREATE FUNCTION dbo.GetEmployeesByDept (@Dept VARCHAR(50))
RETURNS TABLE
AS
RETURN
(
    SELECT EmpID, EmpName, Department, Salary
    FROM Employees
    WHERE Department = @Dept
);

-- How to CALL / USE the function
SELECT * FROM dbo.GetEmployeesByDept('IT');

-- You can also filter further
SELECT * FROM dbo.GetEmployeesByDept('HR')
WHERE Salary > 45000;
```

Output:

```
-- Result of GetEmployeesByDept('IT'):
EmpID | EmpName | Department | Salary
1      | Alice   | IT         | 60000
3      | Carol   | IT         | 70000
```

Key Points to Remember:

- RETURNS TABLE means it gives back a whole table
- No BEGIN...END block needed — just RETURN (SELECT ...)
- Use in FROM clause: SELECT * FROM dbo.FunctionName('value')
- Much faster than Multi-Statement TVF for simple queries

1.3 Multi-Statement Table Valued Function (MTVF)

Definition

A Multi-Statement Table Valued Function (MTVF) returns a TABLE just like an Inline TVF, but it uses multiple SQL statements to build that table step by step. You first define the structure of the output table, then fill it using INSERT statements inside BEGIN...END.

Real-Life Analogy: Think of building a custom report — you first design the report template (columns), then you fill in the data row by row using different rules. An MTVF does exactly this.

When to use MTVF vs Inline TVF:

Inline TVF	Multi-Statement TVF
Simple — one SELECT statement	Complex — multiple SQL steps needed
Faster performance	Slightly slower (table variable used)
No BEGIN...END	Uses BEGIN...END block

Syntax

```
CREATE FUNCTION function_name (@param datatype) RETURNS @TableVariable
TABLE (      Column1 datatype,      Column2 datatype ) AS BEGIN
INSERT INTO @TableVariable      SELECT ... FROM ...      WHERE ...;
RETURN; END;
```

Example: Get Employees with Salary Grade

```
-- MTVF to return employees with a salary grade label
CREATE FUNCTION dbo.GetEmployeesWithGrade (@MinSalary INT)
RETURNS @Result TABLE
(
    EmpName    VARCHAR(100),
    Department VARCHAR(50),
    Salary     INT,
    Grade      VARCHAR(10)
)
AS
BEGIN
    -- Insert employees earning above minimum salary
    INSERT INTO @Result
    SELECT EmpName, Department, Salary,
        CASE
            WHEN Salary >= 65000 THEN 'A'
            WHEN Salary >= 50000 THEN 'B'
            ELSE 'C'
        END AS Grade
    FROM Employees
    WHERE Salary >= @MinSalary;
```



```
RETURN; -- Must have RETURN at the end
END;

-- How to CALL / USE
SELECT * FROM dbo.GetEmployeesWithGrade(45000);
```

Output:

EmpName	Department	Salary	Grade
Alice	IT	60000	B
Carol	IT	70000	A
Dave	Finance	55000	B
Eve	HR	50000	B

Key Points to Remember:

- RETURNS @TableVariable TABLE (...) — defines the output table structure
- You fill the table using INSERT INTO @TableVariable
- End with RETURN; (no value needed — the table is already declared)
- Great for complex logic: IF conditions, multiple INSERTs, loops

2. SQL Joins

A SQL JOIN is used to combine rows from two or more tables based on a related column between them. Joins are one of the most important and frequently tested topics in SQL interviews.

Sample Tables used throughout this section:

Employees Table				Departments Table		
EmpID	EmpName	DeptID	Salary	DeptID	DeptName	Location
1	Alice	10	60000	10	IT	Hyderabad
2	Bob	20	45000	20	HR	Mumbai
3	Carol	10	70000	30	Finance	Delhi
4	Dave	30	55000	40	Marketing	Pune
5	Eve	NULL	50000			

2.1 INNER JOIN

Definition

INNER JOIN returns only the rows that have matching values in BOTH tables. Rows that do not have a match in either table are NOT included in the result. Think of it as the intersection of two sets.

Real-Life Analogy: Imagine two guest lists for a party. INNER JOIN gives you only the people who appear on BOTH lists.

Syntax

```
SELECT columns FROM Table1 INNER JOIN Table2 ON Table1.common_column = Table2.common_column;
```

Example:

```
-- Get employee names with their department name
SELECT E.EmpName, E.Salary, D.DeptName, D.Location
FROM Employees E
INNER JOIN Departments D
ON E.DeptID = D.DeptID;
```

Output:

EmpName	Salary	DeptName	Location
Alice	60000	IT	Hyderabad
Bob	45000	HR	Mumbai
Carol	70000	IT	Hyderabad
Dave	55000	Finance	Delhi

Note: Eve (NULL DeptID) and Marketing (DeptID 40) are NOT shown

2.2 LEFT JOIN (LEFT OUTER JOIN)

Definition

LEFT JOIN returns ALL rows from the LEFT table, and only the matching rows from the RIGHT table. If there is no match in the right table, NULL is shown for the right table columns. The left table data is NEVER lost.

Real-Life Analogy: You have a list of all students (left table). Some students have submitted assignments (right table), some haven't. LEFT JOIN shows ALL students — and marks NULL for those who haven't submitted.

Syntax

```
SELECT columns FROM Table1 LEFT JOIN Table2 ON Table1.common_column =
Table2.common_column;
```

Example:

```
-- Show ALL employees, with department info if available
SELECT E.EmpName, E.Salary, D.DeptName
FROM Employees E
LEFT JOIN Departments D
ON E.DeptID = D.DeptID;
```

Output:

EmpName	Salary	DeptName
Alice	60000	IT
Bob	45000	HR
Carol	70000	IT
Dave	55000	Finance
Eve	50000	NULL

<-- Eve has no department, still shown

2.3 RIGHT JOIN (RIGHT OUTER JOIN)

Definition

RIGHT JOIN returns ALL rows from the RIGHT table, and only the matching rows from the LEFT table. If there is no match in the left table, NULL is shown for the left table columns. The right table data is NEVER lost.

Real-Life Analogy: You have all departments in the company (right table). Some departments have employees (left table), some don't. RIGHT JOIN shows ALL departments — even if no employee belongs to them yet.

Syntax

```
SELECT columns FROM Table1 RIGHT JOIN Table2 ON Table1.common_column =
Table2.common_column;
```

Example:

```
-- Show ALL departments, with employee info if available
SELECT E.EmpName, E.Salary, D.DeptName
```




```
FROM Employees E  
RIGHT JOIN Departments D  
ON E.DeptID = D.DeptID;
```

Output:

EmpName	Salary	DeptName
Alice	60000	IT
Carol	70000	IT
Bob	45000	HR
Dave	55000	Finance
NULL	NULL	Marketing <-- No employee in Marketing, still shown

2.4 FULL OUTER JOIN

Definition

FULL OUTER JOIN returns ALL rows from BOTH tables. Where there is no match, NULL is filled in. It combines the results of both LEFT JOIN and RIGHT JOIN — no data from either table is lost.

Real-Life Analogy: You have two lists — one of all employees, one of all departments. FULL OUTER JOIN shows everything from both lists, filling NULL wherever there is no match.

Syntax

```
SELECT columns FROM Table1 FULL OUTER JOIN Table2 ON  
Table1.common_column = Table2.common_column;
```

Example:

```
-- Show ALL employees AND ALL departments  
SELECT E.EmpName, E.Salary, D.DeptName  
FROM Employees E  
FULL OUTER JOIN Departments D  
ON E.DeptID = D.DeptID;
```

Output:

EmpName	Salary	DeptName
Alice	60000	IT
Bob	45000	HR
Carol	70000	IT
Dave	55000	Finance
Eve	50000	NULL
NULL	NULL	Marketing

<-- No dept for Eve
<-- No employee in Marketing

2.5 CROSS JOIN

Definition

CROSS JOIN returns the Cartesian Product of two tables — meaning every row from the first table is combined with every row from the second table. If Table A has 3 rows and Table B has 4 rows, CROSS JOIN gives $3 \times 4 = 12$ rows. There is NO ON condition.

Real-Life Analogy: Imagine 3 shirt colours and 4 trouser styles. A CROSS JOIN gives you every possible outfit combination — $3 \times 4 = 12$ combinations.

Syntax

```
SELECT columns FROM Table1 CROSS JOIN Table2; -- No ON condition  
needed
```

Example:

```
-- Every employee paired with every department
```

```
SELECT E.EmpName, D.DeptName
FROM Employees E
CROSS JOIN Departments D;

-- Practical use: Generate all test combinations
-- 5 employees x 4 departments = 20 rows total
```

Output:

EmpName	DeptName
Alice	IT
Alice	HR
Alice	Finance
Alice	Marketing
Bob	IT
Bob	HR

... (20 rows total – every combination)

2.6 SELF JOIN

Definition

A SELF JOIN is when a table is joined with ITSELF. This is useful when the table has a column that references another row in the same table — for example, an Employees table where each employee has a ManagerID that refers to another EmpID in the same table.

Real-Life Analogy: Imagine a company hierarchy where every employee has a manager — and the manager is also an employee in the same table. A SELF JOIN helps you connect an employee to their manager's name.

Sample Table for Self Join: EmployeesWithManager

EmpID	EmpName	ManagerID	
1	Alice	NULL	(Alice is the CEO — no manager)
2	Bob	1	(Bob's manager is Alice)
3	Carol	1	(Carol's manager is Alice)
4	Dave	2	(Dave's manager is Bob)

Syntax

```
SELECT A.column, B.column FROM TableName A JOIN TableName B ON
A.foreign_key = B.primary_key; -- A and B are aliases for the same
table
```

Example: Show Each Employee with Their Manager's Name

```
SELECT E.EmpName AS Employee,
       M.EmpName AS Manager
FROM EmployeesWithManager E
LEFT JOIN EmployeesWithManager M
ON E.ManagerID = M.EmpID;

-- LEFT JOIN used so CEO (NULL manager) is also shown
```

Output:

Employee	Manager
Alice	NULL (CEO, no manager)
Bob	Alice
Carol	Alice
Dave	Bob

SQL Joins — Quick Reference Summary

Join Type	What it Returns	Real-Life Example
INNER JOIN	Matching rows from both tables	Students who submitted AND are enrolled
LEFT JOIN	ALL from left + matches from right	All students, NULL if not submitted
RIGHT JOIN	ALL from right + matches from left	All departments, NULL if no employees
FULL OUTER JOIN	ALL rows from both tables	All students + all courses (with NULLs)
CROSS JOIN	Every combination (Cartesian)	All shirt × trouser combinations
SELF JOIN	Table joined with itself	Employee with their manager name

3. Interview Questions & Answers

Study these commonly asked interview questions carefully. Practice explaining these in your own words!

User Defined Functions

Q1. What is a User Defined Function in SQL? How is it different from a Stored Procedure?

Answer: A UDF is a reusable block of code that always RETURNS a value and can be used inside SELECT, WHERE, or ORDER BY. A Stored Procedure may or may not return a value and is called using EXEC — it cannot be used inside a SELECT statement.

Q2. What are the three types of User Defined Functions in SQL Server?

Answer: 1. Scalar Function — returns a single value (e.g., a number or text). 2. Inline Table Valued Function — returns a table using a single SELECT. 3. Multi-Statement Table Valued Function — returns a table built with multiple SQL statements inside BEGIN...END.

Q3. What is the difference between an Inline TVF and a Multi-Statement TVF?

Answer: Inline TVF uses a single SELECT inside RETURN and has no BEGIN...END — it is faster and simpler. MTVF declares a table variable in RETURNS, uses BEGIN...END, and fills it with INSERT statements — it is used when complex logic with multiple steps is needed.

Q4. Can a Scalar UDF be used in a WHERE clause? Give an example.

Answer: Yes! A Scalar UDF can be used anywhere a value is expected. Example: SELECT * FROM Employees WHERE dbo.GetAnnualSalary(Salary) > 600000; — This filters employees whose annual salary (calculated by the UDF) is above 600000.

Q5. Why do we write dbo. before calling a UDF in SQL Server?

Answer: dbo stands for 'database owner' and is the schema name. SQL Server requires you to specify the schema when calling a UDF (unlike built-in functions). If you omit dbo., SQL Server may not find the function and will throw an error.

SQL Joins

Q6. What is the difference between INNER JOIN and LEFT JOIN?

Answer: INNER JOIN returns only rows with matching values in BOTH tables — unmatched rows from either side are excluded. LEFT JOIN returns ALL rows from the LEFT table and only matched rows from the RIGHT table — unmatched right side rows appear as NULL.

**Q7. What is a CROSS JOIN and when would you use it?**

Answer: CROSS JOIN returns the Cartesian product — every row from Table A combined with every row from Table B. No ON condition is needed. It is used to generate all possible combinations, such as all product-colour pairs for a catalogue, or test data generation.

Q8. What is a SELF JOIN? Give a real-world example.

Answer: A SELF JOIN joins a table with itself using aliases. A classic example: an Employees table where each row has a ManagerID that references another EmpID in the same table. By self-joining, you can show each employee alongside their manager's name.

Q9. What is the difference between LEFT JOIN and FULL OUTER JOIN?

Answer: LEFT JOIN returns all rows from the left table and matching rows from the right. FULL OUTER JOIN returns ALL rows from BOTH tables — NULLs appear wherever there is no match on either side. FULL OUTER JOIN is essentially LEFT JOIN + RIGHT JOIN combined.

Q10. How do you find employees who do NOT have a department assigned? Which join would you use?

Answer: Use a LEFT JOIN and filter for NULL: `SELECT E.EmpName FROM Employees E LEFT JOIN Departments D ON E.DeptID = D.DeptID WHERE D.DeptID IS NULL;` — This returns only employees with no matching department record.

Q11. Can you use multiple JOINS in a single query?

Answer: Yes! You can chain multiple JOINS. Example: `SELECT E.EmpName, D.DeptName, L.City FROM Employees E INNER JOIN Departments D ON E.DeptID = D.DeptID INNER JOIN Locations L ON D.LocationID = L.LocationID;` — Each JOIN adds another table to the query.

4. Practice Questions

Try writing the SQL queries yourself before checking! All questions are beginner-friendly and based on the sample Employees and Departments tables used in this document.

User Defined Functions — Practice

1. Create a Scalar UDF called GetMonthlyBonus that accepts a salary and returns 10% of it as a bonus amount. Then call it in a SELECT query to show each employee's bonus.
2. Create an Inline Table Valued Function called GetHighSalaryEmployees that accepts a minimum salary as a parameter and returns all employees earning above that amount.
3. Create a Multi-Statement TVF called GetEmployeeSummary that returns EmpName, Department, Salary, and a new column called 'SalaryCategory' — label salaries above 60000 as 'High', between 45000 and 60000 as 'Medium', and below 45000 as 'Low'.
4. True or False: An Inline TVF uses BEGIN...END and multiple INSERT statements to build its result.
5. What will happen if you try to call a UDF as: SELECT GetAnnualSalary(Salary) FROM Employees; without the dbo. prefix? What is the correct way to call it?
6. Create a Scalar UDF called GetFullDeptLabel that takes a DeptID and returns a formatted string like 'Department: IT | ID: 10'. Test it in a SELECT query.

SQL Joins — Practice

7. Write an INNER JOIN query to show each employee's name, salary, and the name of their department.
8. Write a LEFT JOIN query to display ALL employees including those who do not have a department assigned. Show NULL for the department name if unassigned.
9. Write a RIGHT JOIN query to display ALL departments, even those with no employees. Show the employee count per department using COUNT().
10. Write a FULL OUTER JOIN query that shows all employees and all departments. Identify which employees have no department and which departments have no employees.



11. Using the EmployeesWithManager table (EmpID, EmpName, ManagerID), write a SELF JOIN to display each employee alongside their manager's name. Use LEFT JOIN so the CEO (no manager) is also shown.

12. Suppose you have a Products table (3 rows) and a Colours table (5 rows). How many rows will a CROSS JOIN between them produce? Write the CROSS JOIN query.

13. Write a query using INNER JOIN and WHERE to find employees in the 'IT' department who earn more than 60000.

14. Using LEFT JOIN, write a query to find all employees who do NOT belong to any department (i.e., their DeptID is NULL or has no match in Departments table).

15. Write a query that JOINS Employees, Departments, and a Locations table together to show each employee's name, department, and city.

5. Quick Cheat Sheet

UDF Types at a Glance

UDF Type	Returns	Uses BEGIN...END?	When to Use
Scalar	Single value	Yes	Simple calculations
Inline TVF	Table (1 SELECT)	No	Simple filtering
Multi-Stmt TVF	Table (multi steps)	Yes	Complex logic

Memory Tips

Remember These!

1. Scalar UDF = One answer (like a formula)
1. Inline TVF = Returns a table with one SELECT (no BEGIN...END)
1. Multi-Stmt TVF = Returns a table with complex logic (BEGIN...END + INSERT)
1. INNER JOIN = Intersection (only matched rows)
1. LEFT JOIN = All left + matched right (NULLs on right if no match)
1. RIGHT JOIN = All right + matched left (NULLs on left if no match)
1. FULL OUTER JOIN = Everything from both tables (NULLs where no match)
1. CROSS JOIN = Every row x Every row (Cartesian product, no ON needed)
1. SELF JOIN = Table joined with itself using aliases (for hierarchies)